



<http://algs4.cs.princeton.edu>

2.3 QUICKSORT

- ▶ *quicksort*
- ▶ *selection*
- ▶ *duplicate keys*
- ▶ *system sorts*

Two classic sorting algorithms: mergesort and quicksort

Critical components in the world's computational infrastructure.

- Full scientific understanding of their properties has enabled us to develop them into practical system sorts.
- Quicksort honored as one of top 10 algorithms of 20th century in science and engineering.

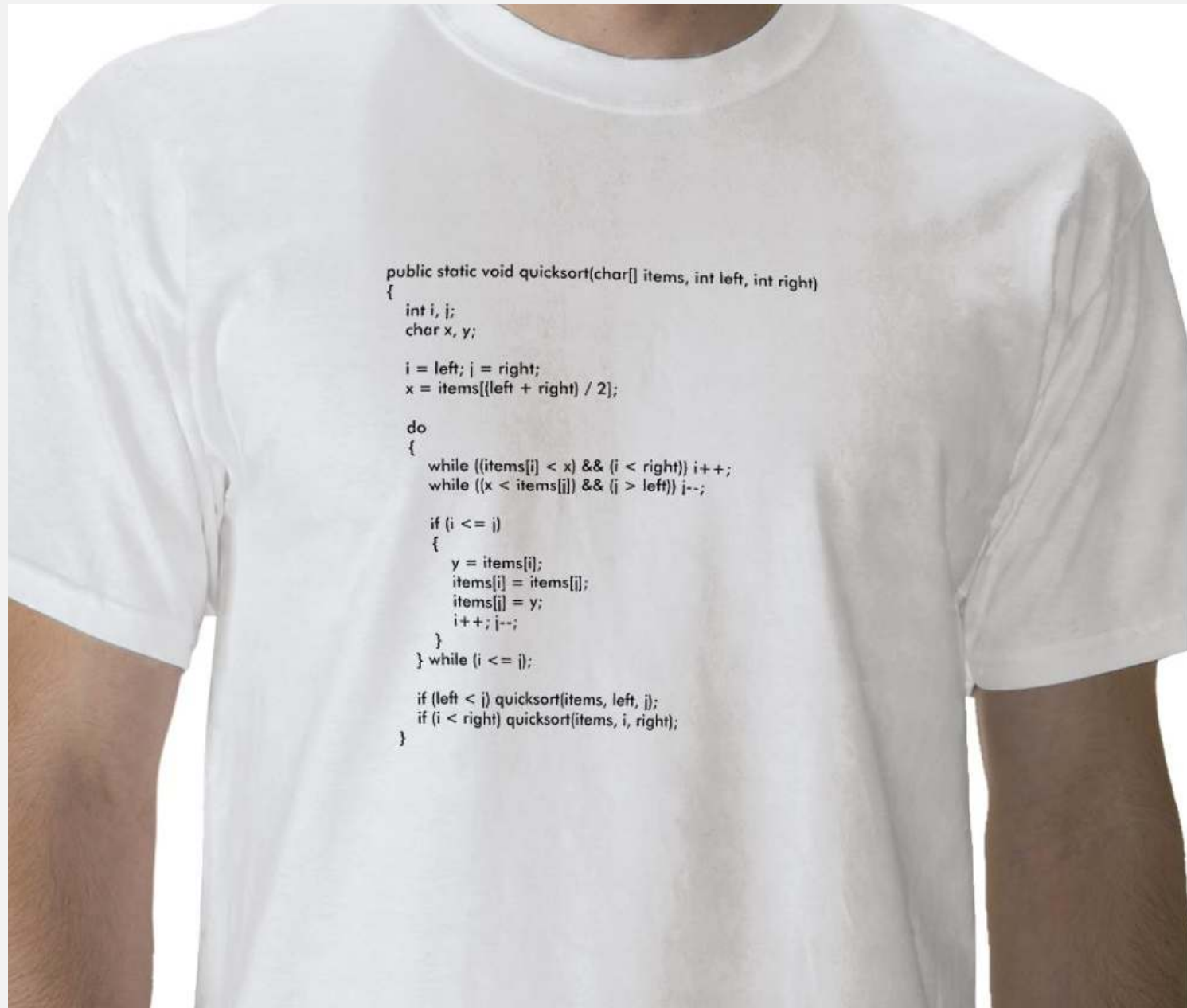
Mergesort. [last lecture]



Quicksort. [this lecture]



Quicksort t-shirt

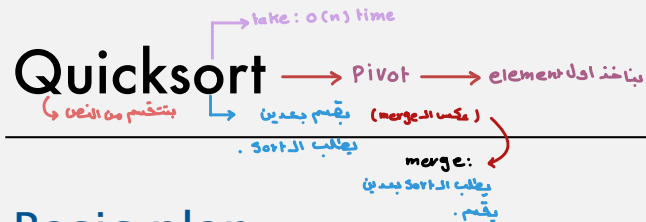




<http://algs4.cs.princeton.edu>

2.3 QUICKSORT

- ▶ *quicksort*
- ▶ *selection*
- ▶ *duplicate keys*
- ▶ *system sorts*



Basic plan.

- **Shuffle** the array.
- **Partition** so that, for some j
 - entry $a[j]$ is in place
 - no larger entry to the left of j
 - no smaller entry to the right of j
- **Sort** each subarray recursively.

* L : less
 E : equal
 G : Greater
 وبتقسيم الأعداد
 إلى : الأصغر و الأكبر
 وبتقسيم مرة ثانية إلى جزء الأقل
 و جزء الأكبر .

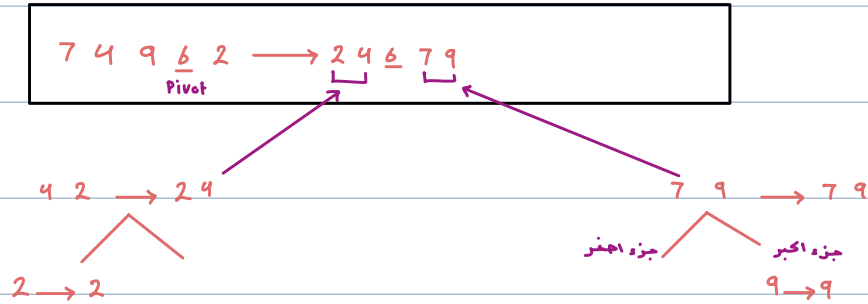


— راجع تقسيم أبجدياً . A B C D E F G H I J K L M N O P Q R S T U V W X Y Z .

input	Q	U	I	C	K	S	O	R	T	E	X	A	M	P	L	E
shuffle	K	R	A	T	E	L	E	P	U	I	M	Q	C	X	O	S
partition	E	C	A	I	E	K	L	P	U	T	M	Q	R	X	O	S
sort left	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
sort right	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X
result	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X

partitioning item
 not greater
 not less

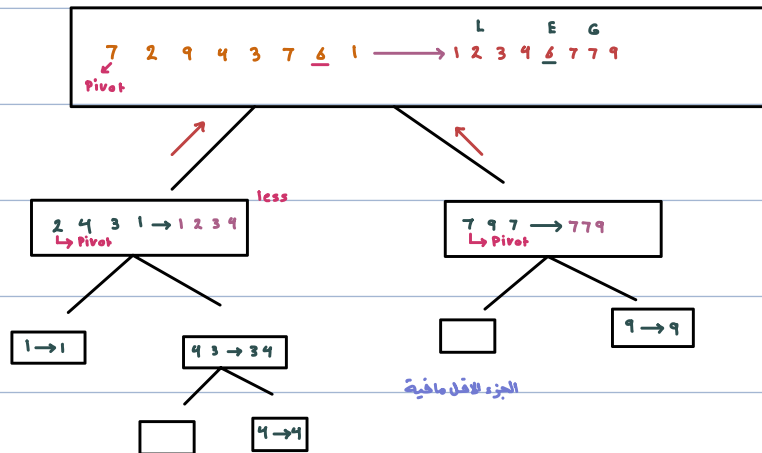
Join :



Execution Example :

Pivot selection

جزء اصغر
2 4 3 1
جزء اكبر
6 7 9 7

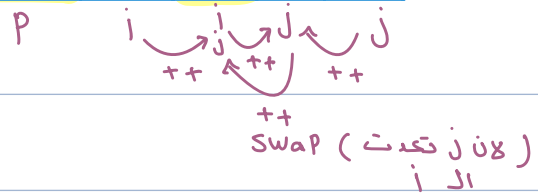
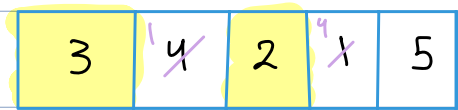
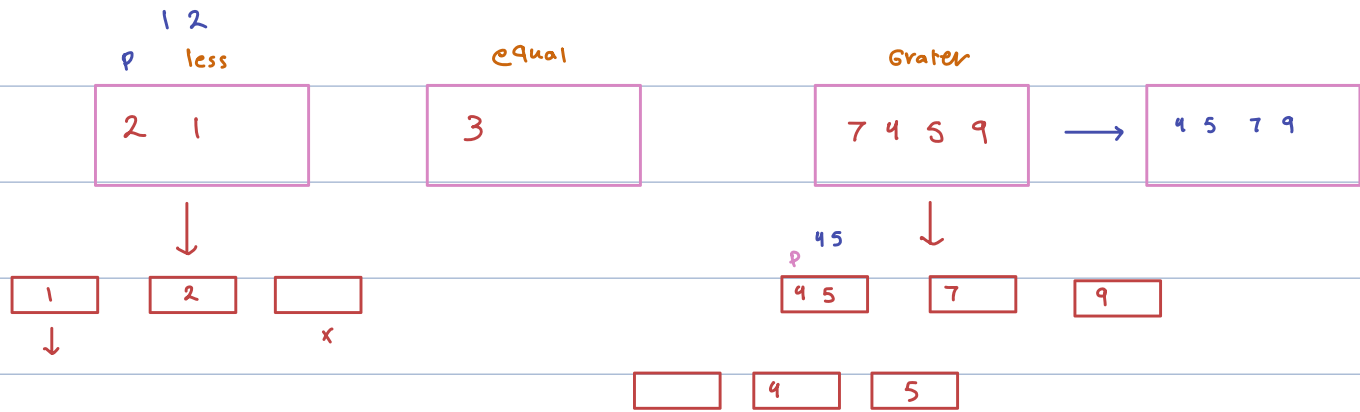
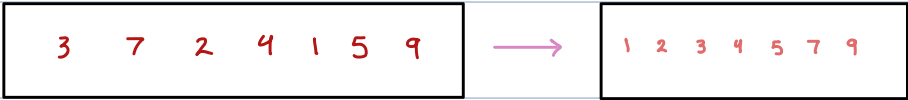


choice of Pivot :

1- Chosen as a Pivot and then swapped with the first key.

يمكن اختيار ال Pivot ما في مكان في list وابداً مع ال Pivot الى موجود عندي او

اخيرة ما ضمن ال element ما بختياره حسب يتطلع من ضمن قائمة المتساوية.



final:



مكانها الصحيح
~~~~~

# Tony Hoare

---

- Invented quicksort to translate Russian into English.
  - [ but couldn't explain his algorithm or implement it! ]
- Learned Algol 60 (and recursion).
- Implemented quicksort.



Tony Hoare  
1980 Turing Award



ALGORITHM 64  
QUICKSORT  
C. A. R. HOARE  
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

```
procedure quicksort (A,M,N); value M,N;  
    array A; integer M,N;  
comment Quicksort is a very fast and convenient method of  
sorting an array in the random-access store of a computer. The  
entire contents of the store may be sorted, since no extra space is  
required. The average number of comparisons made is  $2(M-N) \ln$   
 $(N-M)$ , and the average number of exchanges is one sixth this  
amount. Suitable refinements of this method will be desirable for  
its implementation on any actual computer;  
begin    integer I,J;  
        if M < N then begin partition (A,M,N,I,J);  
                        quicksort (A,M,J);  
                        quicksort (A, I, N)  
        end  
end  
quicksort
```

Communications of the ACM (July 1961)



# Tony Hoare

---

- Invented quicksort to translate Russian into English.
  - [ but couldn't explain his algorithm or implement it! ]
- Learned Algol 60 (and recursion).
- Implemented quicksort.



**Tony Hoare**  
**1980 Turing Award**

*“ There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult. ”*

*“ I call it my billion-dollar mistake. It was the invention of the null reference in 1965... This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years. ”*

- Refined and popularized quicksort.
- Analyzed quicksort.



Bob Sedgewick

Programming  
Techniques

S. L. Graham, R. L. Rivest  
Editors

## Implementing Quicksort Programs

Robert Sedgewick  
Brown University

This paper is a practical study of how to implement the Quicksort sorting algorithm and its best variants on real computers, including how to apply various code optimization techniques. A detailed implementation combining the most effective improvements to Quicksort is given, along with a discussion of how to implement it in assembly language. Analytic results describing the performance of the programs are summarized. A variety of special situations are considered from a practical standpoint to illustrate Quicksort's wide applicability as an internal sorting method which requires negligible extra storage.

Key Words and Phrases: Quicksort, analysis of algorithms, code optimization, sorting

CR Categories: 4.0, 4.6, 5.25, 5.31, 5.5

Acta Informatica 7, 327—355 (1977)  
© by Springer-Verlag 1977

## The Analysis of Quicksort Programs\*

Robert Sedgewick

Received January 19, 1976

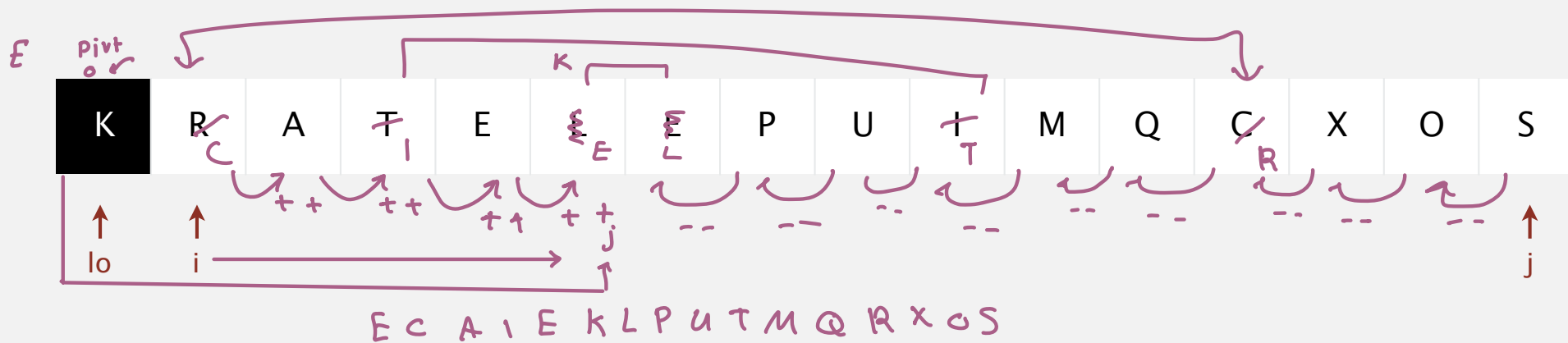
*Summary.* The Quicksort sorting algorithm and its best variants are presented and analyzed. Results are derived which make it possible to obtain exact formulas describing the total expected running time of particular implementations on real computers of Quicksort and an improvement called the median-of-three modification. Detailed analysis of the effect of an implementation technique called loop unwrapping is presented. The paper is intended not only to present results of direct practical utility, but also to illustrate the intriguing mathematics which arises in the complete analysis of this important algorithm.

# Quicksort partitioning demo

Repeat until  $i$  and  $j$  pointers cross.

- Scan  $i$  from left to right so long as  $(a[i] < a[lo])$ .
- Scan  $j$  from right to left so long as  $(a[j] > a[lo])$ .
- Exchange  $a[i]$  with  $a[j]$ .

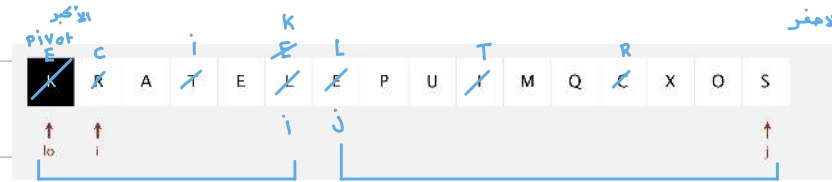
A B C D E F G H I J K L M N O P Q  
R S T U V W X Y Z.



★ Pivot:  $i < \text{Pivot} \rightarrow \checkmark i++$   
 $i < \text{Pivot} \rightarrow \times \text{go to } j$

$j > \text{Pivot} \rightarrow \checkmark j--$   
 $j > \text{Pivot} \rightarrow \times \text{Switch between } i \text{ and } j.$   
↪ 151  
until:  $i \geq j$   
Pivot swap  $j$ .

A B c D E F G H i j k L M N o P Q R S T U V W x Y Z.



# Quicksort partitioning demo

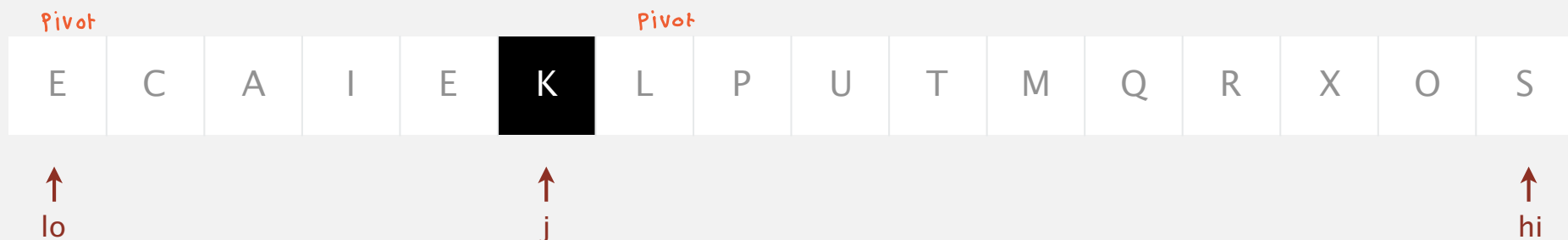
---

Repeat until  $i$  and  $j$  pointers cross.

- Scan  $i$  from left to right so long as  $(a[i] < a[lo])$ .
- Scan  $j$  from right to left so long as  $(a[j] > a[lo])$ .
- Exchange  $a[i]$  with  $a[j]$ .

When pointers cross.

- Exchange  $a[lo]$  with  $a[j]$ .



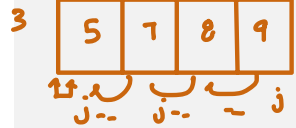
**partitioned!**

# Quicksort: Java code for partitioning

worst C.S

best C.S

Sorted reverse indantikal key.



q-5  
Partition  
O(n<sup>2</sup>)

```
private static int partition(Comparable[] a, int lo, int hi)
{
    int i = lo, j = hi+1;
    while (true)
    {
        while (less(a[++i], a[lo]))
        {
            if (i == hi) break;
        }
        while (less(a[lo], a[--j]))
        {
            if (j == lo) break;
        }

        if (i >= j) break;
        exch(a, i, j);
    }

    exch(a, lo, j);
    return j;
}
```

find item on left to swap

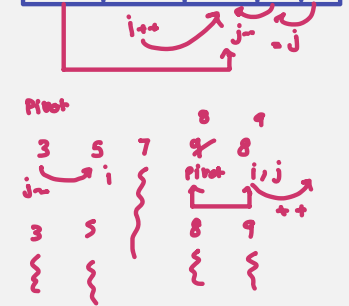
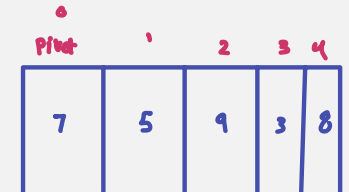
find item on right to swap

check if pointers cross

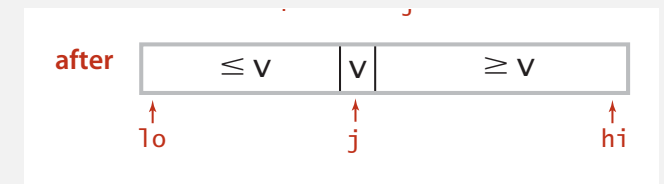
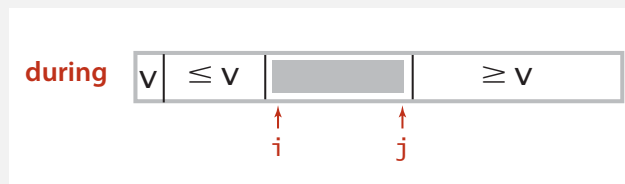
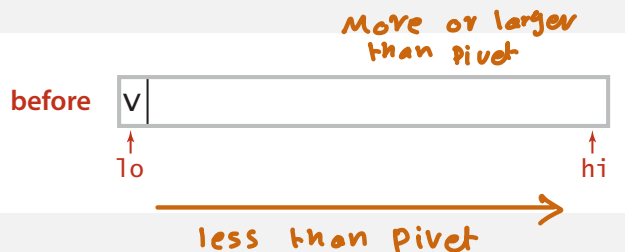
swap

swap with partitioning item

return index of item now known to be in place



n=5  
n tests  
n-1 tests  
Partitions: 2 ~ 5  
log<sub>2</sub> n  
run time: O(n log<sub>2</sub> n)



# Quicksort: Java implementation

---

```
public class Quick
{
    private static int partition(Comparable[] a, int lo, int hi)
    { /* see previous slide */ }

    public static void sort(Comparable[] a)
    {
        StdRandom.shuffle(a);
        sort(a, 0, a.length - 1);
    }

    private static void sort(Comparable[] a, int lo, int hi)
    {
        if (hi <= lo) return;
        int j = partition(a, lo, hi);
        sort(a, lo, j-1);
        sort(a, j+1, hi);
    }
}
```

← shuffle needed for  
performance guarantee  
(stay tuned)

# Quicksort trace

Pivot ↗ element بر اول  
↘ element بر آخر

|                | lo | j  | hi | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|----------------|----|----|----|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| initial values |    |    |    | Q | U | I | C | K | S | O | R | T | E | X  | A  | M  | P  | L  | E  |
| random shuffle |    |    |    | K | R | A | T | E | L | E | P | U | I | M  | Q  | C  | X  | O  | S  |
|                | 0  | 5  | 15 | E | C | A | I | E | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 0  | 3  | 4  | E | C | A | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 0  | 2  | 2  | A | C | E | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 0  | 0  | 1  | A | C | E | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 1  |    | 1  | A | C | E | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 4  |    | 4  | A | C | E | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 6  | 6  | 15 | A | C | E | E | I | K | L | P | U | T | M  | Q  | R  | X  | O  | S  |
|                | 7  | 9  | 15 | A | C | E | E | I | K | L | M | O | P | T  | Q  | R  | X  | U  | S  |
|                | 7  | 7  | 8  | A | C | E | E | I | K | L | M | O | P | T  | Q  | R  | X  | U  | S  |
|                | 8  |    | 8  | A | C | E | E | I | K | L | M | O | P | T  | Q  | R  | X  | U  | S  |
|                | 10 | 13 | 15 | A | C | E | E | I | K | L | M | O | P | S  | Q  | R  | T  | U  | X  |
|                | 10 | 12 | 12 | A | C | E | E | I | K | L | M | O | P | R  | Q  | S  | T  | U  | X  |
|                | 10 | 11 | 11 | A | C | E | E | I | K | L | M | O | P | Q  | R  | S  | T  | U  | X  |
|                | 10 |    | 10 | A | C | E | E | I | K | L | M | O | P | Q  | R  | S  | T  | U  | X  |
|                | 14 | 14 | 15 | A | C | E | E | I | K | L | M | O | P | Q  | R  | S  | T  | U  | X  |
|                | 15 |    | 15 | A | C | E | E | I | K | L | M | O | P | Q  | R  | S  | T  | U  | X  |
| result         |    |    |    | A | C | E | E | I | K | L | M | O | P | Q  | R  | S  | T  | U  | X  |

no partition for subarrays of size 1

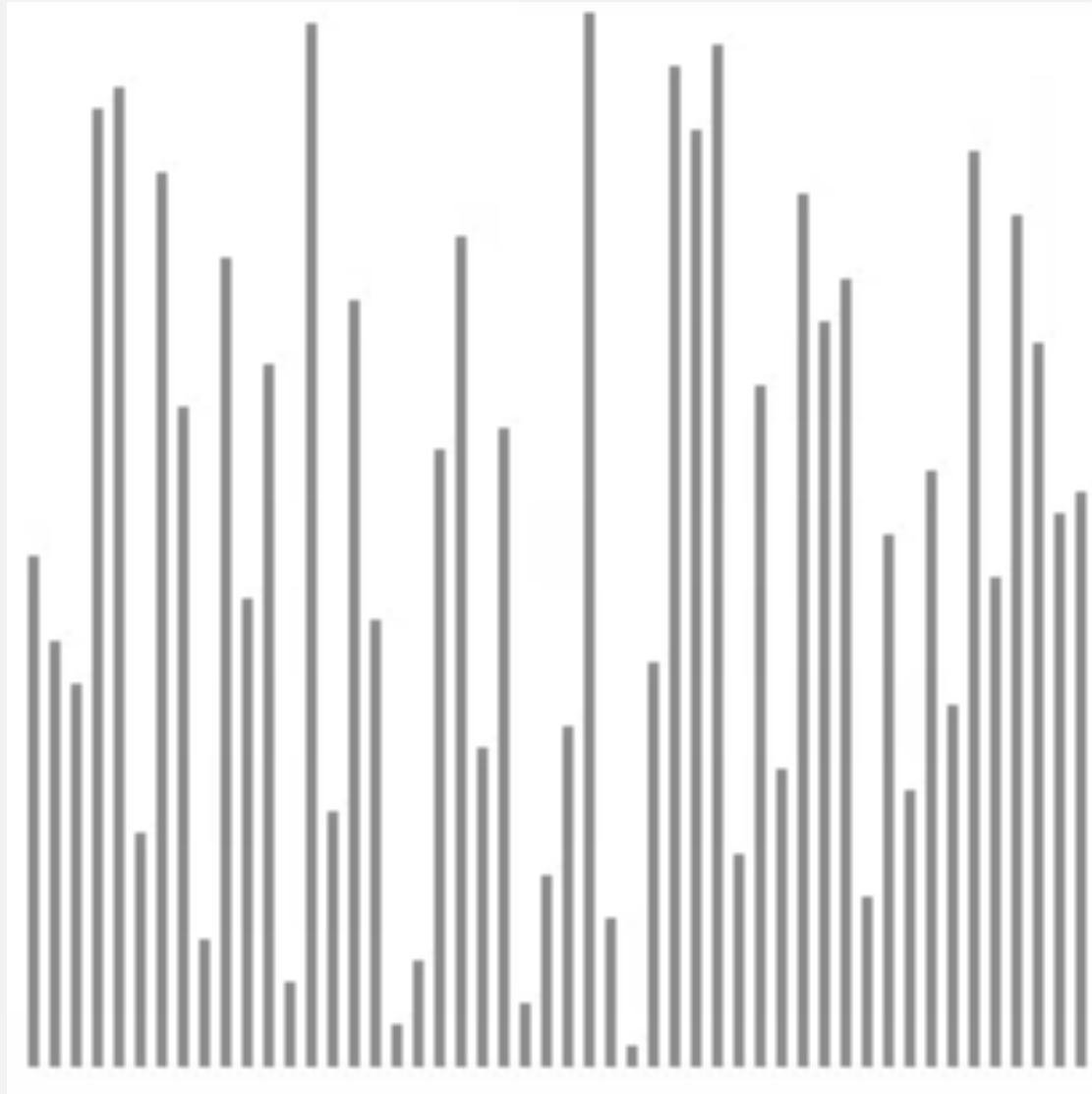
Quicksort trace (array contents after each partition)



# Quicksort animation

---

50 random items



<http://www.sorting-algorithms.com/quicksort>

- ▲ algorithm position
- in order
- current subarray
- not in order

## Quicksort: implementation details

---

**Partitioning in-place.** Using an extra array makes partitioning easier (and **stable**), but is not worth the cost.

**Terminating the loop.** Testing whether the pointers cross is trickier than it might seem.

 **Equal keys.** When duplicates are present, it is (counter-intuitively) better to stop scans on keys equal to the partitioning item's key.

**Preserving randomness.** Shuffling is needed for performance guarantee.

**Equivalent alternative.** Pick a random partitioning item in each subarray.

# Quicksort: empirical analysis (1961)

## Running time estimates:

- Algol 60 implementation.
- National-Elliott 405 computer.

**Table 1**

| NUMBER OF ITEMS | MERGE SORT    | QUICKSORT    |
|-----------------|---------------|--------------|
| 500             | 2 min 8 sec   | 1 min 21 sec |
| 1,000           | 4 min 48 sec  | 3 min 8 sec  |
| 1,500           | 8 min 15 sec* | 5 min 6 sec  |
| 2,000           | 11 min 0 sec* | 6 min 47 sec |

\* These figures were computed by formula, since they cannot be achieved on the 405 owing to limited store size.

sorting N 6-word items with 1-word keys



**Elliott 405 magnetic disc  
(16K words)**

# Quicksort: empirical analysis

## Running time estimates:

- Home PC executes  $10^8$  compares/second.
- Supercomputer executes  $10^{12}$  compares/second.

| computer | insertion sort ( $N^2$ ) |           |           | mergesort ( $N \log N$ ) |          |         | quicksort ( $N \log N$ ) |         |         |
|----------|--------------------------|-----------|-----------|--------------------------|----------|---------|--------------------------|---------|---------|
|          | thousand                 | million   | billion   | thousand                 | million  | billion | thousand                 | million | billion |
| home     | instant                  | 2.8 hours | 317 years | instant                  | 1 second | 18 min  | instant                  | 0.6 sec | 12 min  |
| super    | instant                  | 1 second  | 1 week    | instant                  | instant  | instant | instant                  | instant | instant |

**Lesson 1.** Good algorithms are better than supercomputers.

**Lesson 2.** Great algorithms are better than good ones.

# Quicksort: best-case analysis

Perfectly balanced  
التقسيم مادل

Best case. Number of compares is  $\sim N \lg N$ .

★ Expected Running Time:

Good call: the sizes of L and G are each less than  $3s/4$ .

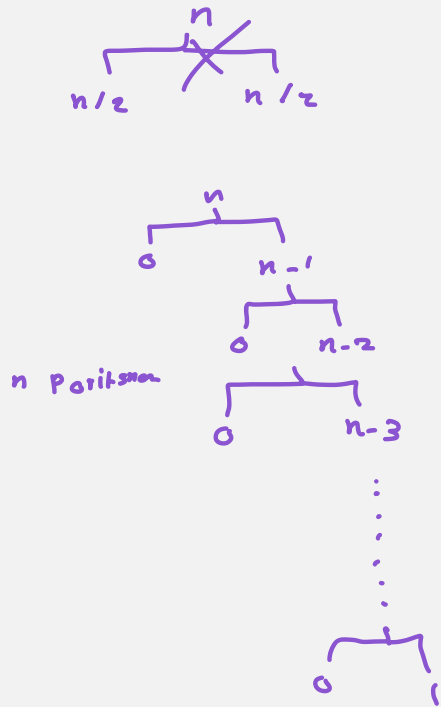
bad call: one of L and G has size greater than  $3s/4$ .

|                |    |    | a[ ] |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|----------------|----|----|------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| lo             | j  | hi | 0    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| initial values |    |    | H    | A | C | B | F | E | G | D | L | I | K  | J  | N  | M  | O  |
| random shuffle |    |    | H    | A | C | B | F | E | G | D | L | I | K  | J  | N  | M  | O  |
| 0              | 7  | 14 | D    | A | C | B | F | E | G | H | L | I | K  | J  | N  | M  | O  |
| 0              | 3  | 6  | B    | A | C | D | F | E | G | H | L | I | K  | J  | N  | M  | O  |
| 0              | 1  | 2  | A    | B | C | D | F | E | G | H | L | I | K  | J  | N  | M  | O  |
| 0              |    | 0  | A    | B | C | D | F | E | G | H | L | I | K  | J  | N  | M  | O  |
| 2              |    | 2  | A    | B | C | D | F | E | G | H | L | I | K  | J  | N  | M  | O  |
| 4              | 5  | 6  | A    | B | C | D | E | F | G | H | L | I | K  | J  | N  | M  | O  |
| 4              |    | 4  | A    | B | C | D | E | F | G | H | L | I | K  | J  | N  | M  | O  |
| 6              |    | 6  | A    | B | C | D | E | F | G | H | L | I | K  | J  | N  | M  | O  |
| 8              | 11 | 14 | A    | B | C | D | E | F | G | H | J | I | K  | L  | N  | M  | O  |
| 8              | 9  | 10 | A    | B | C | D | E | F | G | H | I | J | K  | L  | N  | M  | O  |
| 8              |    | 8  | A    | B | C | D | E | F | G | H | I | J | K  | L  | N  | M  | O  |
| 10             |    | 10 | A    | B | C | D | E | F | G | H | I | J | K  | L  | N  | M  | O  |
| 12             | 13 | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |
| 12             |    | 12 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |
| 14             |    | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |
|                |    |    | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |

# Quicksort: **worst-case analysis**

unbalanced → Sorted  
 اگر چنبره بزرگ و کوچک  
 بزرگتر  
 If I choose Pivot the  
 minimum or maximum.  
 کم یا بیشترین  
 در آید  
 $n^2$ , sorted

Worst case. Number of compares is  $\sim \frac{1}{2} N^2$ .



|                |    |    | a[ ] |   |   |   |   |   |   |   |   |   |    |    |    |    |    |  |  |
|----------------|----|----|------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|--|--|
| lo             | j  | hi | 0    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |  |  |
| initial values |    |    | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| random shuffle |    |    | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 0              | 0  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 1              | 1  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 2              | 2  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 3              | 3  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 4              | 4  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 5              | 5  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 6              | 6  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 7              | 7  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 8              | 8  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 9              | 9  | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 10             | 10 | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 11             | 11 | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 12             | 12 | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 13             | 13 | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
| 14             |    | 14 | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |
|                |    |    | A    | B | C | D | E | F | G | H | I | J | K  | L  | M  | N  | O  |  |  |

## Quicksort: average-case analysis

**Proposition.** The average number of compares  $C_N$  to quicksort an array of  $N$  distinct keys is  $\sim 2N \ln N$  (and the number of exchanges is  $\sim \frac{1}{3} N \ln N$ ).

*↪ number of compares*

**Pf.**  $C_N$  satisfies the recurrence  $C_0 = C_1 = 0$  and for  $N \geq 2$ :

$$C_N = \overset{\text{partitioning}}{\downarrow} (N+1) + \left( \frac{C_0 + C_{N-1}}{N} \right) + \overset{\text{left}}{\downarrow} \left( \frac{C_1 + C_{N-2}}{N} \right) + \overset{\text{right}}{\downarrow} \dots + \left( \frac{C_{N-1} + C_0}{N} \right)$$

*↖ partitioning probability*

- Multiply both sides by  $N$  and collect terms:

$$NC_N = N(N+1) + 2(C_0 + C_1 + \dots + C_{N-1})$$

- Subtract from this equation the same equation for  $N-1$ :

$$NC_N - (N-1)C_{N-1} = 2N + 2C_{N-1}$$

- Rearrange terms and divide by  $N(N+1)$ :

$$\frac{C_N}{N+1} = \frac{C_{N-1}}{N} + \frac{2}{N+1}$$

# Quicksort: average-case analysis

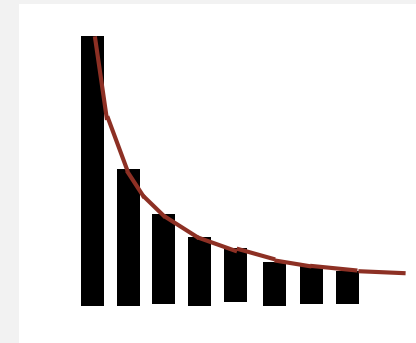
- Repeatedly apply above equation:

$$\begin{aligned}\frac{C_N}{N+1} &= \frac{C_{N-1}}{N} + \frac{2}{N+1} \\ &= \frac{C_{N-2}}{N-1} + \frac{2}{N} + \frac{2}{N+1} \quad \leftarrow \text{substitute previous equation} \\ &= \frac{C_{N-3}}{N-2} + \frac{2}{N-1} + \frac{2}{N} + \frac{2}{N+1} \\ &= \frac{2}{3} + \frac{2}{4} + \frac{2}{5} + \dots + \frac{2}{N+1}\end{aligned}$$

previous equation

- Approximate sum by an integral:

$$\begin{aligned}C_N &= 2(N+1) \left( \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \dots + \frac{1}{N+1} \right) \\ &\sim 2(N+1) \int_3^{N+1} \frac{1}{x} dx\end{aligned}$$



- Finally, the desired result:

$$C_N \sim 2(N+1) \ln N \approx 1.39N \lg N$$



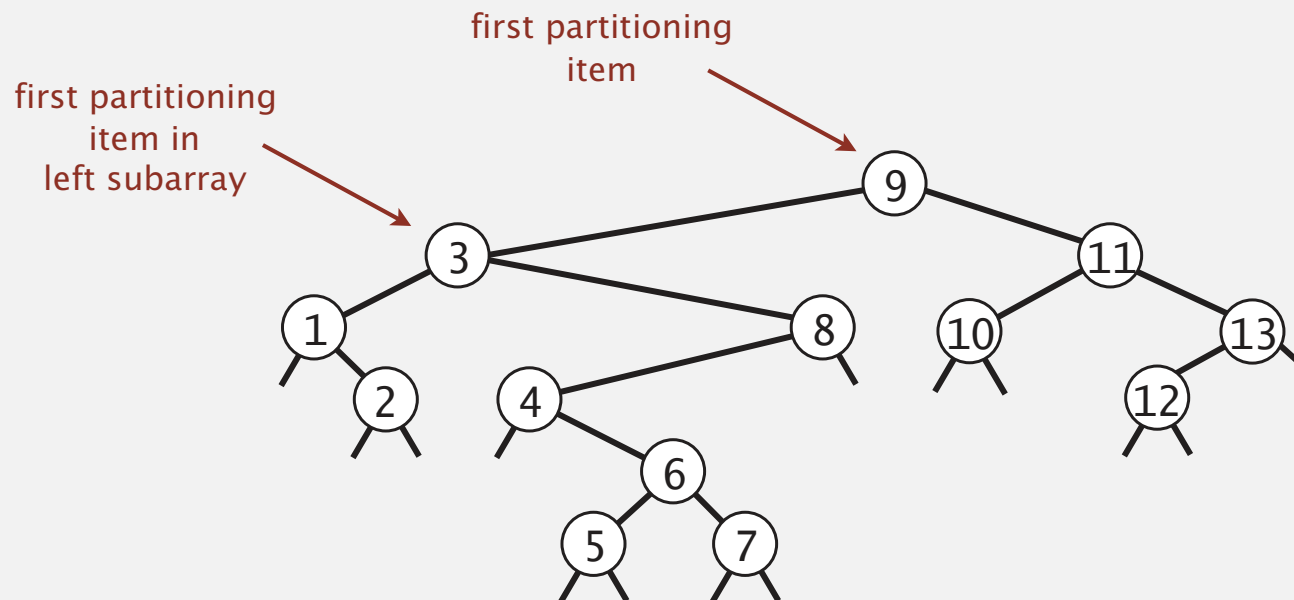
# Quicksort: average-case analysis

**Proposition.** The average number of compares  $C_N$  to quicksort an array of  $N$  distinct keys is  $\sim 2N \ln N$  (and the number of exchanges is  $\sim \frac{1}{3} N \ln N$ ).

**Pf 2.** Consider BST representation of keys 1 to  $N$ .

shuffle

|   |    |   |   |   |   |   |   |    |    |    |   |   |
|---|----|---|---|---|---|---|---|----|----|----|---|---|
| 9 | 10 | 2 | 5 | 8 | 7 | 6 | 1 | 11 | 12 | 13 | 3 | 4 |
|---|----|---|---|---|---|---|---|----|----|----|---|---|



# Quicksort: average-case analysis

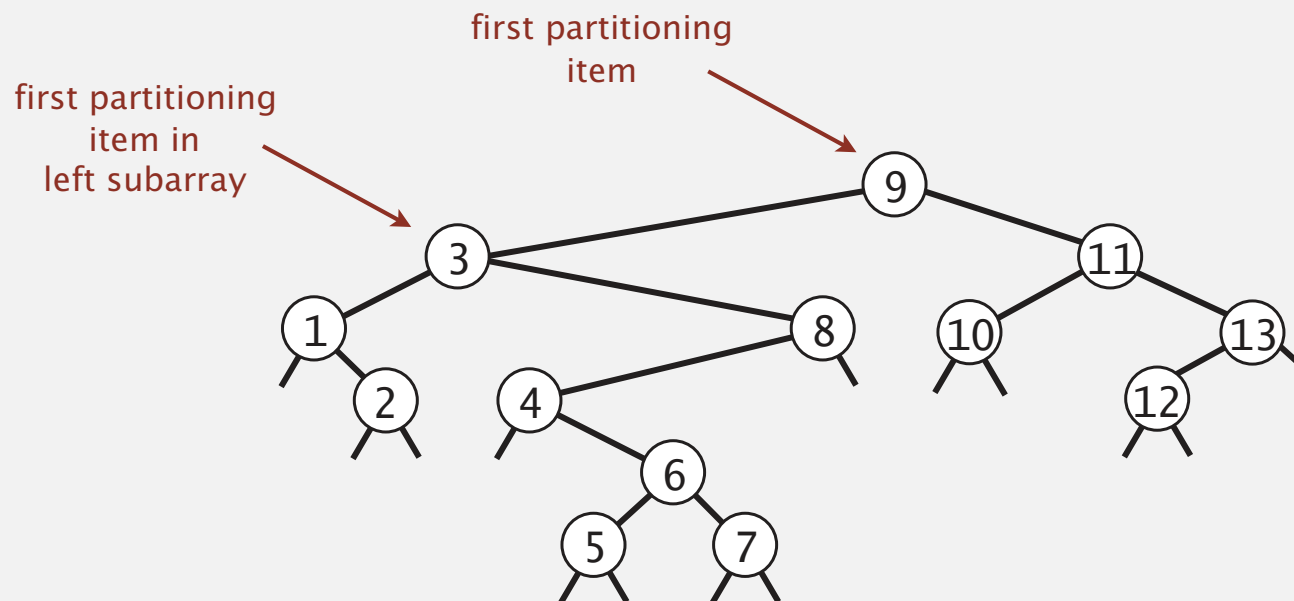
**Proposition.** The average number of compares  $C_N$  to quicksort an array of  $N$  distinct keys is  $\sim 2N \ln N$  (and the number of exchanges is  $\sim \frac{1}{3} N \ln N$ ).

**Pf 2.** Consider BST representation of keys 1 to  $N$ .

- A key is compared only with its ancestors and descendants.
- Probability  $i$  and  $j$  are compared equals  $2 / |j - i + 1|$ .

3 and 6 are compared  
(when 3 is partition)

1 and 6 are not compared  
(because 3 is partition)



## Quicksort: average-case analysis

---

**Proposition.** The average number of compares  $C_N$  to quicksort an array of  $N$  distinct keys is  $\sim 2N \ln N$  (and the number of exchanges is  $\sim \frac{1}{3} N \ln N$ ).

**Pf 2.** Consider BST representation of keys 1 to  $N$ .

- A key is compared only with its ancestors and descendants.
- Probability  $i$  and  $j$  are compared equals  $2 / |j - i + 1|$ .

- Expected number of compares =  $\sum_{i=1}^N \sum_{j=i+1}^N \frac{2}{j-i+1} = 2 \sum_{i=1}^N \sum_{j=2}^{N-i+1} \frac{1}{j}$   
  
all pairs  $i$  and  $j$   
 $\leq 2N \sum_{j=1}^N \frac{1}{j}$   
 $\sim 2N \int_{x=1}^N \frac{1}{x} dx$   
 $= 2N \ln N$

# Quicksort: summary of performance characteristics

---

Quicksort is a (Las Vegas) **randomized algorithm**.

- Guaranteed to be correct.
- Running time depends on random shuffle.

**Average case.** Expected number of compares is  $\sim 1.39 N \lg N$ .

- 39% more compares than mergesort.
- **Faster than mergesort** in practice because of less data movement.

**Best case.** Number of compares is  $\sim N \lg N$ .

**Worst case.** Number of compares is  $\sim \frac{1}{2} N^2$ .

[ but more likely that lightning bolt strikes computer during execution ]



# Quicksort properties

**Proposition.** Quicksort is an **in-place** sorting algorithm.

ميتبقى بنفس المكان

not stable

**Pf.**

- Partitioning: constant extra space.
- Depth of recursion: logarithmic extra space (with high probability).

can guarantee logarithmic depth by recurring on smaller subarray before larger subarray (requires using an explicit stack)

**Proposition.** Quicksort is **not stable**.

**Pf.** [ by counterexample ]

| i | j | 0              | 1              | 2              | 3              |
|---|---|----------------|----------------|----------------|----------------|
|   |   | B <sub>1</sub> | C <sub>1</sub> | C <sub>2</sub> | A <sub>1</sub> |
| 1 | 3 | B <sub>1</sub> | C <sub>1</sub> | C <sub>2</sub> | A <sub>1</sub> |
| 1 | 3 | B <sub>1</sub> | A <sub>1</sub> | C <sub>2</sub> | C <sub>1</sub> |
| 0 | 1 | A <sub>1</sub> | B <sub>1</sub> | C <sub>2</sub> | C <sub>1</sub> |



<http://algs4.cs.princeton.edu>

## 2.3 QUICKSORT

---

- ▶ *quicksort*
- ▶ *selection*
- ▶ *duplicate keys*
- ▶ *system sorts*

# Duplicate keys

---

Often, purpose of sort is to bring items with equal keys together.

- Sort population by age.
- Remove duplicates from mailing list.
- Sort job applicants by college attended.

Typical characteristics of such applications.

- Huge array.
- Small number of key values.

من اوله الى list : j  
يبدأ من الـ Pivot : k  
ننا

```
Chicago 09:25:52
Chicago 09:03:13
Chicago 09:21:05
Chicago 09:19:46
Chicago 09:19:32
Chicago 09:00:00
Chicago 09:35:21
Chicago 09:00:59
Houston 09:01:10
Houston 09:00:13
Phoenix 09:37:44
Phoenix 09:00:03
Phoenix 09:14:25
Seattle 09:10:25
Seattle 09:36:14
Seattle 09:22:43
Seattle 09:10:11
Seattle 09:22:54
```

↑  
key

# Duplicate keys

$O(n^2)$  runtime

Quicksort with **duplicate keys**. Algorithm can go **quadratic** unless partitioning stops on equal keys!



**Caveat emptor.** Some textbook (and commercial) implementations go quadratic when many duplicate keys.



What is the result of partitioning the following array?

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | A | A | A | A | A | A | A | A | A | A | A | A | A | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

there is no less .

A.

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | A | A | A | A | A | A | A | A | A | A | A | A | A | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

B.

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | A | A | A | A | A | A | A | A | A | A | A | A | A | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

C.

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | A | A | A | A | A | A | A | A | A | A | A | A | A | A | A |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Partitioning an array with all equal keys

|   |    | a[ ] |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |
|---|----|------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| i | j  | 0    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|   |    | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 1 | 15 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 1 | 15 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 2 | 14 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 2 | 14 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 3 | 13 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 3 | 13 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 4 | 12 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 4 | 12 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 5 | 11 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 5 | 11 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 6 | 10 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 6 | 10 | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 7 | 9  | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
| 7 | 9  | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
|   | 8  | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |
|   | 8  | A    | A | A | A | A | A | A | A | A | A | A  | A  | A  | A  | A  | A  |

## Duplicate keys: the problem

---

**Recommended.** Stop scans on items equal to the partitioning item.

**Consequence.**  $\sim N \lg N$  compares when all keys equal.

B A A B A B C C B C B

A A A A A A A A A A A

**Mistake.** Don't stop scans on items equal to the partitioning item.

**Consequence.**  $\sim \frac{1}{2} N^2$  compares when all keys equal.

الوضع سوء

B A A B A B B B C C C

A A A A A A A A A A A

**Desirable.** Put all items equal to the partitioning item in place.

A A A B B B B B C C C

A A A A A A A A A A A

# Sorting summary

|             | inplace?         | stable? | best                  | average           | worst             | remarks                                                                                 |
|-------------|------------------|---------|-----------------------|-------------------|-------------------|-----------------------------------------------------------------------------------------|
| selection   | ✓                |         | $\frac{1}{2} N^2$     | $\frac{1}{2} N^2$ | $\frac{1}{2} N^2$ | $N$ exchanges                                                                           |
| insertion   | ✓                | ✓       | $N$                   | $\frac{1}{4} N^2$ | $\frac{1}{2} N^2$ | use for small $N$<br>or partially ordered                                               |
| shell       | ✓                |         | $N \log_3 N$          | ?                 | $c N^{3/2}$       | tight code;<br>subquadratic <span style="color: red;">not included</span>               |
| merge       |                  | ✓       | $\frac{1}{2} N \lg N$ | $N \lg N$         | $N \lg N$         | $N \log N$ guarantee;<br>stable → يعكس                                                  |
| timsort     |                  | ✓       | $N$                   | $N \lg N$         | $N \lg N$         | improves mergesort<br>when preexisting order                                            |
| quick       | $\Theta(n)$<br>✓ |         | $N \lg N$             | $2 N \ln N$       | $\frac{1}{2} N^2$ | $N \log N$ probabilistic guarantee;<br>fastest in practice                              |
| 3-way quick | ✓                |         | $N$                   | $2 N \ln N$       | $\frac{1}{2} N^2$ | improves quicksort<br>when duplicate keys <span style="color: red;">not included</span> |
| ?           | ✓                | ✓       | $N$                   | $N \lg N$         | $N \lg N$         | holy sorting grail                                                                      |

# System sort in Java 7

---

## `Arrays.sort()`.

- Has method for objects that are Comparable.
- Has overloaded method for each primitive type.
- Has overloaded method for use with a Comparator.
- Has overloaded methods for sorting subarrays.



## Algorithms.

- Dual-pivot quicksort for primitive types.
- Timsort for reference types.

Q. Why use different algorithms for primitive and reference types?

## Summary of Sorting Algorithms

| Algorithm      | Time                      | Notes                         |
|----------------|---------------------------|------------------------------------------------------------------------------------------------------------------|
| selection-sort | $O(n^2)$                  | <ul style="list-style-type: none"><li>▪ in-place</li><li>▪ slow (good for small inputs)</li></ul>                |
| insertion-sort | $O(n^2)$                  | <ul style="list-style-type: none"><li>▪ in-place</li><li>▪ slow (good for small inputs)</li></ul>                |
| quick-sort     | $O(n \log n)$<br>expected | <ul style="list-style-type: none"><li>▪ in-place, randomized</li><li>▪ fastest (good for large inputs)</li></ul> |
| heap-sort      | $O(n \log n)$             | <ul style="list-style-type: none"><li>▪ in-place</li><li>▪ fast (good for large inputs)</li></ul>                |
| merge-sort     | $O(n \log n)$             | <ul style="list-style-type: none"><li>▪ sequential data access</li><li>▪ fast (good for huge inputs)</li></ul>   |

SWaP:

|           | inplace? | stable? | best                          | average  | worst    |
|-----------|----------|---------|-------------------------------|----------|----------|
| selection | ✓        |         | $O(n)$                        | $O(n)$   | $O(n)$   |
| insertion | ✓        | ✓       | compare $O(n)$<br>swaP $O(1)$ | $O(n^2)$ | $O(n^2)$ |